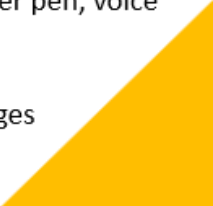


UI design

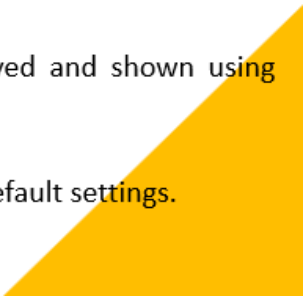
Principle 1: Allow users to maintain control

- **Define interactive modes that allow the user to let them do whatever they want freely**
 - Define different modes and allows easy switching between modes
 - E.g., spell check and editing modes in the word processor, read mode, browser modes, etc.
 - **Allow flexible interaction**
 - Allow users to interact via multiple ways like the mouse, keyboard, digitizer pen, voice inputs, multitouch inputs, etc.
 - **Allow user interaction to be uninterruptable and undoable**
 - Freely allows the user to quit a work and do another without losing changes
 - Undo/redo also allowed
- 

Principle 1: Allow users to maintain control

- **Allow creating customized operations**
 - Create macros for Frequently occurring operations(e.g., like in MS word)
 - For advanced users to customize UI
 - **Hide technical internals from casual users**
 - User is unaware of the underlying OS, file management operations and other complexities
 - E.g., in Application software, there is no need to give OS commands.
 - **Design for direct interaction with on-screen objects**
 - Icons or other objects could be dragged or scaled up or down as if they exist physically.
- 

Principle 2: Reduce User's memory load

- **Well designed system doesn't test user's memory**
 - E.g., Use list boxes or checkbox inputs rather than text boxes.
 - **Reduce demand to learn more**
 - Users should not have to recall past inputs, rather they are saved and shown using visuals.
 - **Establish meaningful defaults**
 - User should have a preference to customize layout or change the default settings.
 - However, reset could change them to defaults again.
- 

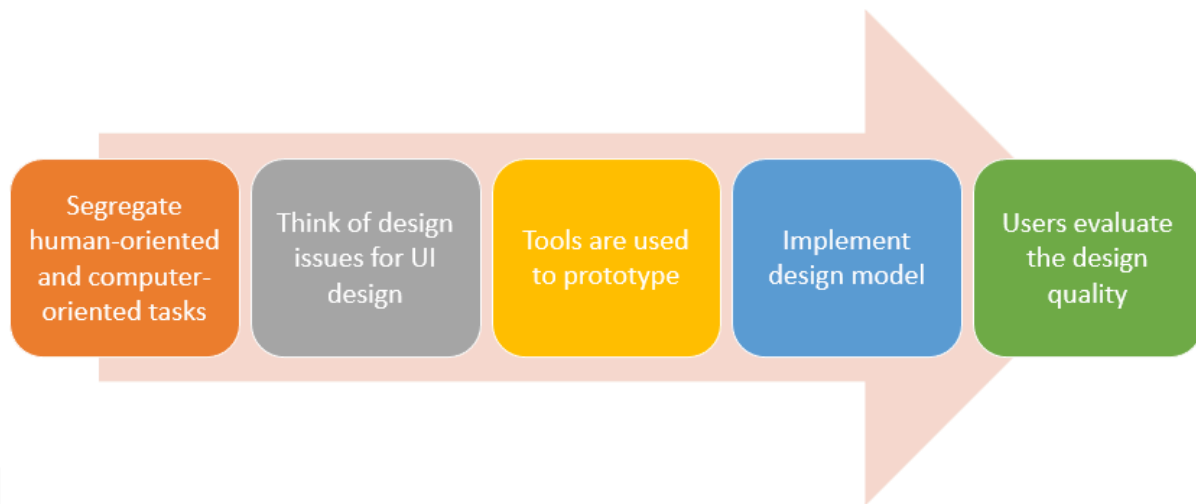
Principle 2: Reduce User's memory load

- **Define intuitive shortcuts**
 - Bind shortcuts that are easy to remember like ctrl+p for **print**
 - **Visual layout should be near to real-time systems**
 - E.g., the bill paying process should be the same as in real life.
 - All steps must be fulfilled.
 - **Disclose information in a progressive fashion**
 - Information displayed on multiple levels.
 - E.g., E-commerce websites have just pictures. Click that to show detailed information, different underline options shown when clicked on the list further
- 

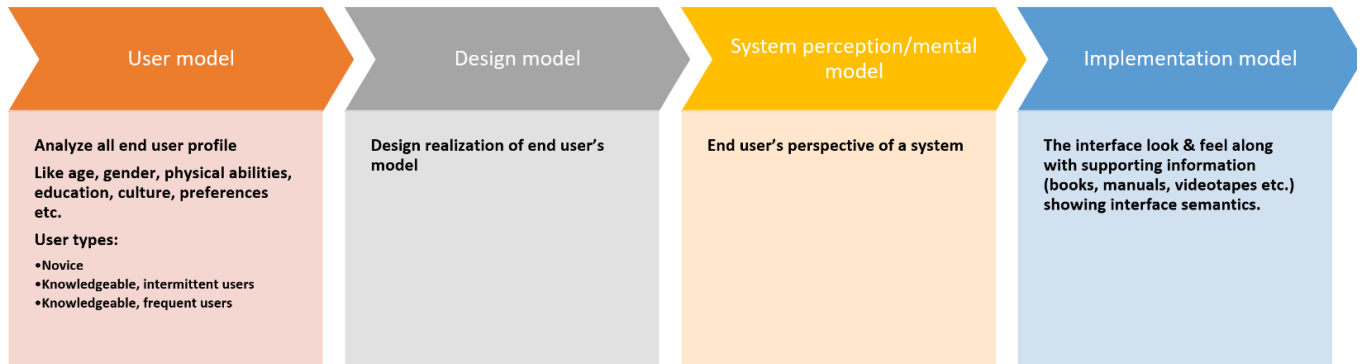
Principle 3: Make consistent interface

- **Allow users to put tasks in meaningful context**
 - User should have an idea of the work they are doing
 - **Like windows should be labeled with titles, consistent color coding should be used for the same tasks etc.**
- **Maintain consistency across a family of applications**
 - A set of applications must follow a particular pattern
 - As the front and the back view is same in whole MS office.
- **Don't make changes in system against user's previous habits, until you are compelled to do so**
 - Don't redefine the user's past experiences in UI
 - For example, Ctrl+S is used for save. Don't bind it with scaling. As user is expecting on his past experience that it'll save the file.

UI Analysis & design



UI Design models



1. User Model

This is the "**Who.**" Before a single pixel is drawn, the designer must create a profile of the target audience.

- **User Profiles:** You analyze demographics like age, education, and even physical abilities (e.g., color blindness or motor skills).
- **User Types:**
 - **Novice:** Needs wizards, prompts, and clear labels because they don't know the system.
 - **Knowledgeable, Intermittent:** Knows the basics but forgets specifics; needs good menus and "recognition over recall."
 - **Knowledgeable, Frequent:** Power users who want shortcuts, hotkeys, and fast workflows.

2. Design Model

This is the "**Plan.**" It is the designer's vision of how the system should work based on the User Model.

- It incorporates the data structures, the flow of screens, and the "rules" of the interface.
- **The Goal:** To realize the requirements of the user in a way that is technically feasible.

3. System Perception (Mental Model)

This is the "**User's Reality**." It is the image the user forms in their head about how the system works while they are using it.

- If a user clicks a "Save" icon and expects a file to be on their desktop, but it actually goes to the cloud, their **Mental Model** is broken.
- A good UI makes it easy for the user's mental model to match the Designer's Model.

4. Implementation Model

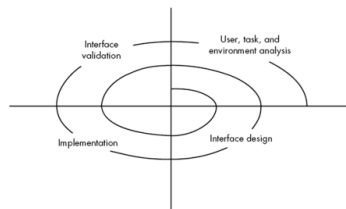
This is the "**Actual Product**." It is the final version that the user touches and sees.

- **Look & Feel:** The colors, fonts, and animations.
- **Semantics:** The "meaning" of the interface—what happens when you swipe, click, or long-press.
- **Supporting Info:** This includes the "Help" section, manuals, or tutorials that help the user understand the system if the UI isn't immediately intuitive.

User Analysis questions for better user categorization

- Are **users trained professionals, technician, clerical, or manufacturing workers**?
- **Level of formal education** of an average user?
- Are the users capable of learning from written materials or have they expressed a desire for classroom training?
- users **expert typists / keyboard phobic**?
- **age range** of the user community?
- User's **gender**?
- How are users compensated for the work they perform?
- Working hours: **normal office hours / until the job is done**?
- **Software usage frequency** ? (mostly/occasionally)
- **primary spoken language among users**?
- **consequences of a user's mistake while using the system**?
- Are users experts in the subject matter that is addressed by the system?
- Are users interested in learning about the technology that sits behind the interface?

UI analysis & design Process



- task analysis:
 - Detailed task analysis done after user analysis.
 - The tasks that users will perform to accomplish system goals.
- User environment analysis:
 - done to check if the current physical environment be able to cater the system needs?
- Output: Analysis model for interface

Interface Design Steps



Interface Analysis

- Task Analysis & Modelling

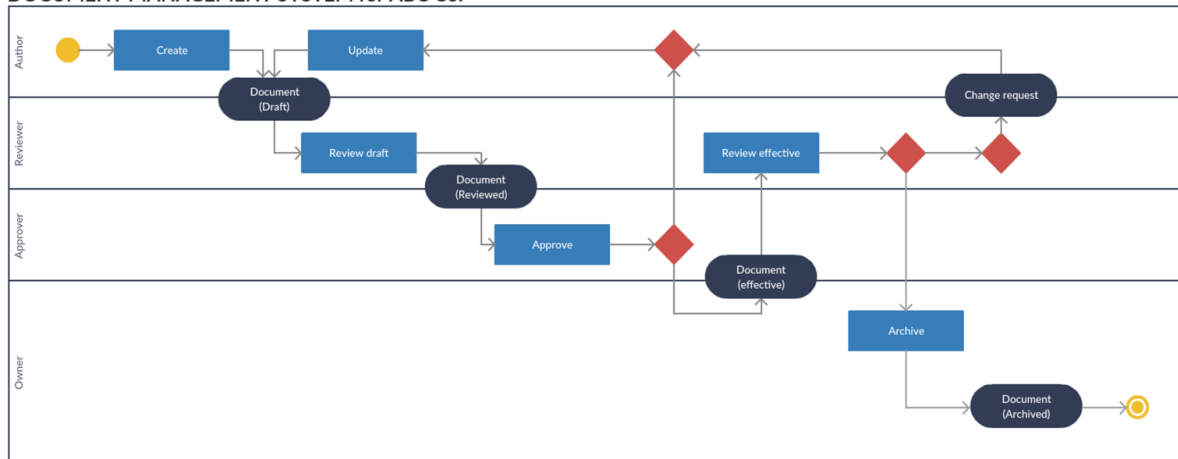
- answer the given questions ...
 - Task identification in specific circumstances.
 - Tasks and sub tasks performed by user
 - sequence of work tasks—the workflow
- Use-cases:
 - define basic interaction of actor and system
 - written using informal paragraphs.
 - Can be used to derive tasks, sub tasks and interfaces
- Task elaboration:
 - Stepwise elaboration or task refinement of interactive tasks
 - Derive either manually or use a preexisting system to identify them
- Object elaboration:
 - extract physical objects from system to define their classes and behavior
- Workflow analysis:
 - How defines how a work process is
 - a work process is completed when several people (and roles) are involved
 - Shown using UML swim line diagram

Interface Analysis

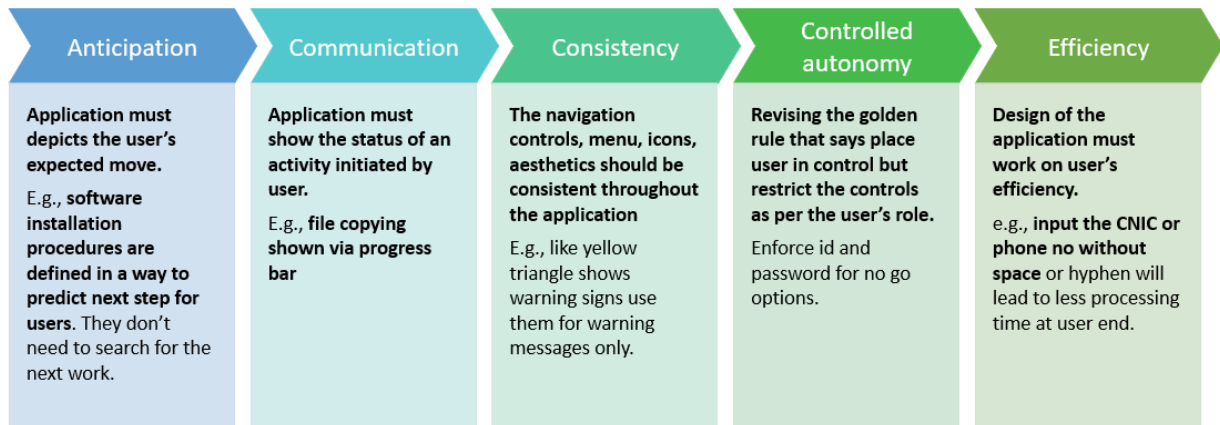
- Task Analysis & Modelling

Swim line diagram

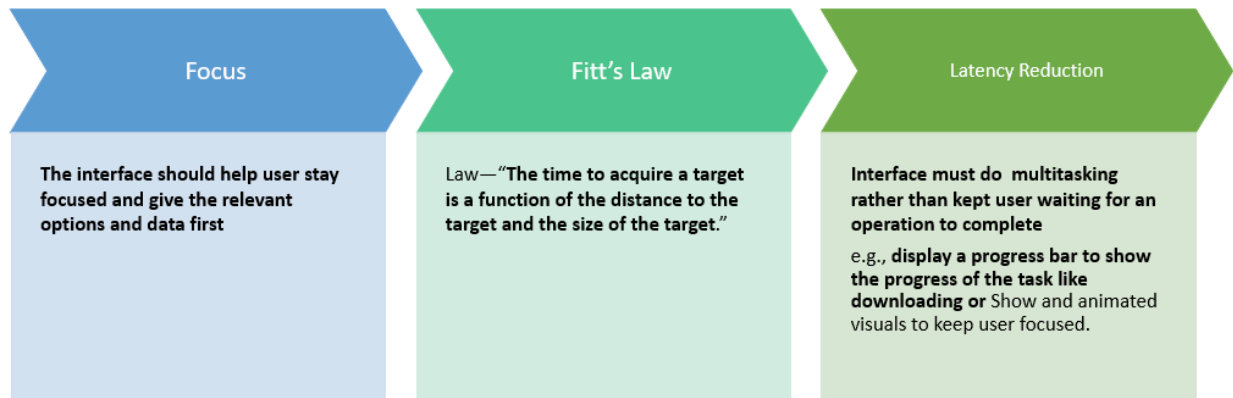
DOCUMENT MANAGEMENT SYSTEM for ABC Co.



Revised Interface Design Guidelines



Revised Interface Design Guidelines



✓ 1. Anticipation

The system should **predict what the user wants to do next.**

◆ Explanation:

- Design the interface so **users don't have to think** too much.
- **Guide** them **step-by-step**.

♦ **Example:**

- During software installation:
 - "Next → Accept → Install" flow is obvious.
- User doesn't need to search for the next action.

👉 **Goal: Reduce user effort and confusion.**

2. Communication

The system should always **inform the user about what is happening**.

♦ **Explanation:**

- Show **status of ongoing actions**.
- **Provide feedback** for every **user action**.

♦ **Example:**

- File copying shows a **progress bar**
- "Uploading... 60% complete"

👉 **Goal: Keep users informed and reduce uncertainty.**

3. Consistency

Everything in the UI should behave and **look uniform**.

Navigation controls, menu, icons, aesthetics shd be consistehnt thru out the application

♦ **Explanation:**

- **Same icons, colors, layouts should mean the same thing everywhere.**

♦ **Example:**

-  **Yellow triangle** always means **warning**
- Same button style across all pages

👉 **Goal:** Make the system predictable and **easy to learn**.

4. Controlled Autonomy

Users should have control, but **within safe limits**.

♦ **Explanation:**

- Allow freedom, but **restrict dangerous or unauthorized actions**.

♦ **Example:**

- **Require password for sensitive actions**
- Disable restricted options for certain roles

👉 **Goal:** Balance **freedom + security**

5. Efficiency

The system should **help users complete tasks quickly.**

♦ Explanation:

- **Minimize unnecessary steps**
- **Reduce typing and effort**

♦ Example:

- Enter CNIC/phone number without spaces or hyphens → faster processing

👉 **Goal:** Save time and effort.

6. Focus

Keep the user **focused on important tasks.**

♦ Explanation:

- **Show** only **relevant options**
- **Avoid clutter** and distractions

♦ Example:

- Dashboard shows **key data first**
- Hide advanced options unless needed

👉 **Goal:** **Improve concentration and usability.**

7. Fitt's Law

The **time to click something {acquire a target}** depends on:

- **Distance to the target**
- **Size of the target**

♦ **Explanation:**

- **Bigger and closer buttons** are **easier** and **faster to click**

♦ **Example:**

- Large "Submit" button placed near user's cursor area

👉 **Goal:** Make interaction faster and easier.

8. Latency Reduction

Reduce waiting time or **make waiting feel shorter**.

♦ **Explanation:**

- **Perform tasks in background** (multitasking)
- **Keep user engaged** during delays

♦ **Example:**

- **Progress bars** during downloads
- **Animations** or **loading indicators**

👉 **Goal:** Improve perceived performance and user experience.

Revised Interface Design Guidelines for a Web App

Learnability	• Minimize learning time by creating simple & intuitive designs
Maintain work product integrity	• An interface should autosaved the user data • e.g., filling out long forms and losing data in case of an error in an input field. This leads to frustration. So the data should be saved for all the correct fields.
Readability	• All information presented should be readable by all age group users
Track state	• Cookies can be used to track user activities so user may logout any time & can continue from the same state after logging in again
Visible navigation	• Obvious navigation bars mentioned on every interface of the application.

1. Learnability

The interface should be **easy to learn for** new users.

- Users should understand how to use the system quickly.
- **Avoid complex layouts** or **confusing features**.

♦ Example:

- Simple signup/login process
- Clear buttons like "Next", "Submit"

👉 **Goal:** **Minimize learning time** and make the system intuitive.

2. Maintain Work Product Integrity

User data should **not be lost**, even if something goes wrong.

- The system should **automatically save (autosave) user input**.
- Prevent frustration caused by data loss.
- While filling a long form:
 - If an error occurs, already filled correct fields remain saved
- Google Docs auto-saving your work

👉 **Goal: Protect user effort and data.**

3. Readability

Content should be **easy to read for all users**.

- **Use clear fonts, proper spacing, and simple language.**
- Make it **accessible** for **different age groups**.
- Large readable text
- Good contrast (dark text on light background)

👉 **Goal:** Improve understanding and user comfort.

4. Track State

The system should **remember what the user was doing**.

- **Track user activity** so they can **continue later**.
- Often done using **cookies** or **session storage**.
- You log out of a website → log back in → continue from where you left

- Shopping cart items remain saved

👉 **Goal:** Provide **continuity and convenience**.

✓ 5. Visible Navigation

Navigation should be **clear and always accessible**.

- Users should **always know where they are and where to go next**.
- **Navigation menus should be visible on all pages**.
- Top navigation bar (Home, About, Contact)
- Sidebar menus in dashboards

👉 **Goal:** Help users move easily through the application

Interface Design Workflow:

- Derive and refine information from requirements
 - Develop rough layout sketch
 - Map user objective to actions (like Home, Contact, booking, facilities, etc....)
 - Create storyboard screen images for every interface
 - Refine layouts by improving aesthetics
 - Develop an activity diagram to show the design flow
 - Develop state diagrams to show changes in state after transitions
 - Describe interface layout for every state
 - Refine & review the model.
- 

Interface Design Evolution

